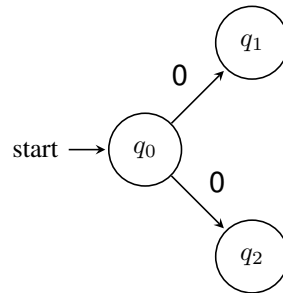


1 Non-deterministic Finite Automata

Non-deterministic computation:



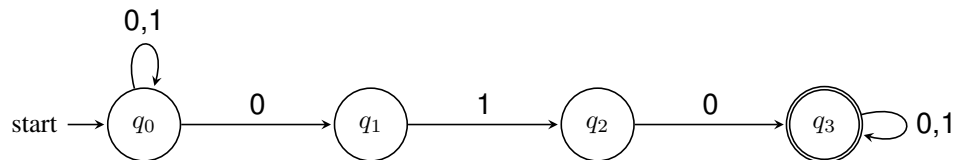
For a given state and input, transition function can have more than one results, or no result at all.

$$\delta(q_0, 0) = \{q_1, q_2\}$$

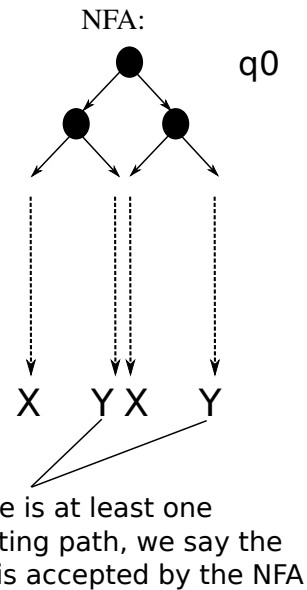
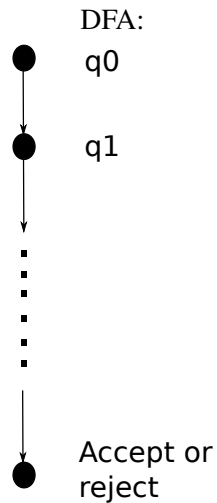
$$\delta(q_1, 1) = \emptyset$$

An NFA example:

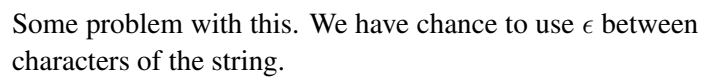
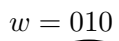
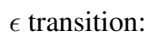
$L : \{w | w \text{ contains } 010 \text{ as a substring}\}$



eg. $w = 00010$:



eg. $w = 00010 \in L(N)$

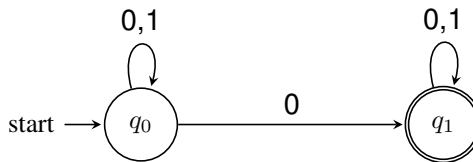


Definition: An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

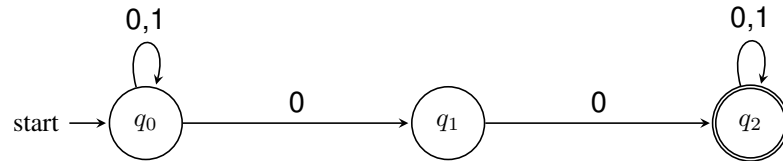
- Q is a finite set of states.
- Σ is a finite alphabet.
- $\delta : Q \times \Sigma_\epsilon \rightarrow 2^Q$ is the transition function. ($\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$)
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of accept states.

Examples:

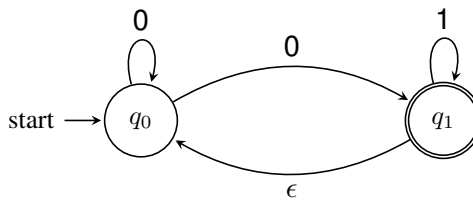
- $L = \{w \mid w \text{ contains } 0\}$



- $L = \{w \mid w \text{ contains } 00 \text{ as a substring}\}$



- $L = \{w \mid w \text{ begins with } 0\}$

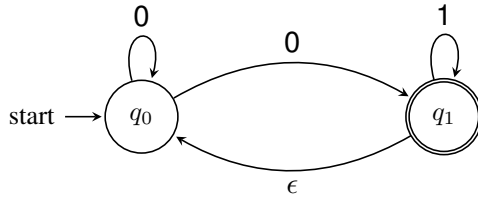


1.1 Equivalence of DFA & NFA

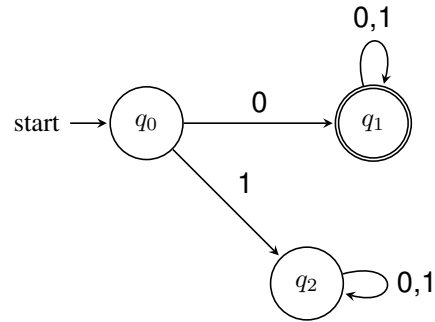
Definition: Two automata M_1 and M_2 are *equivalent* if $L(M_1) = L(M_2)$, denoted as $M_1 \equiv M_2$.

$L = \{w \mid w \text{ starts with } 0\}$. $M \equiv N$, $L(M) = L(N)$.

N:



M:

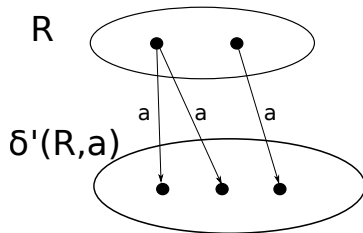


Theorem 1.1 For every NFA, there exists an equivalent DFA.

Proof By construction. For a given NFA $N=(Q, \Sigma, \delta, q_0, F)$, construct DFA $M=(Q', \Sigma, \delta', q'_0, F')$. First, assume no ϵ -transitions.

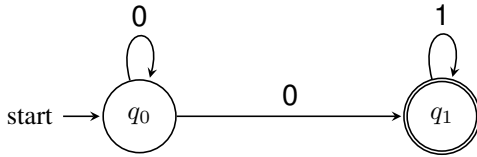
- $Q' = 2^Q$
- $q'_0 = q_0$
- For $R \in Q'$, and $a \in \Sigma$, let $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ for some } r \in R\}$. If R is a state of M , it is also a set of states of N . When M reads a symbol a in state R , it shows where a takes each state in R . Because each state may go to a set of states, we take the union of all these sets.

Note: $\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$ (union of all states which we can go from R 's elements).

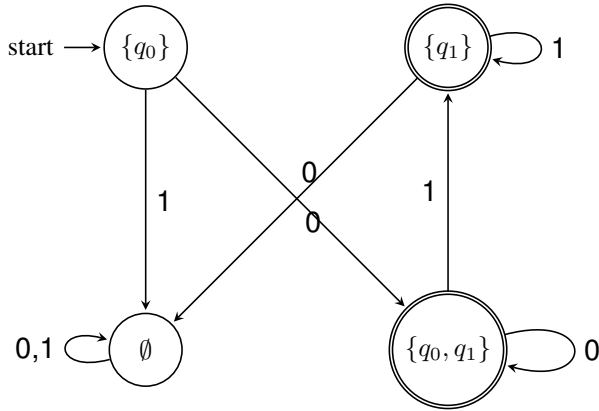


- $F' = \{R \in Q' \mid R \text{ contains some accept state from } F\}$

Example:



$$\delta(q_0, 1) = \emptyset, \delta(q_0, 0) = \{q_0, q_1\}, \delta(q_1, 1) = \{q_1\}, \delta(q_1, 0) = \emptyset.$$



Any state which has q_1 in it is an accept state.

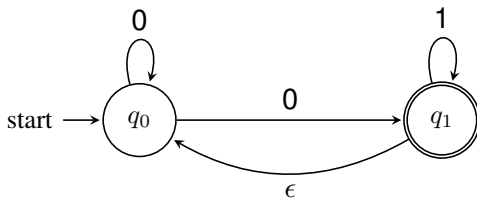
1.1.1 Lazy construction:

NFA	0	1
$\rightarrow 1$	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
*9	6,8	5

$\Rightarrow$

DFA	0	1
$\rightarrow \{1\}$	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
*{1,3,7,9}	{2,4,6,8}	{5}
*{1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

Now, consider the ϵ -transitions. For $R \subseteq Q$, let:
 $E(R) = \{q | q \text{ can be reached from } R \text{ by traveling along 0 or more } \epsilon \text{ arrows}\}$
 which is called the ϵ -closure of R . E.g.



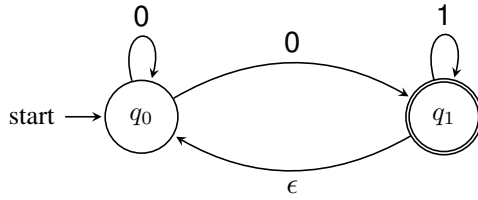
$$E(\{q_0\}) = \{q_0\}, E(\{q_1\}) = \{q_0, q_1\}$$

Now we modify our construction:

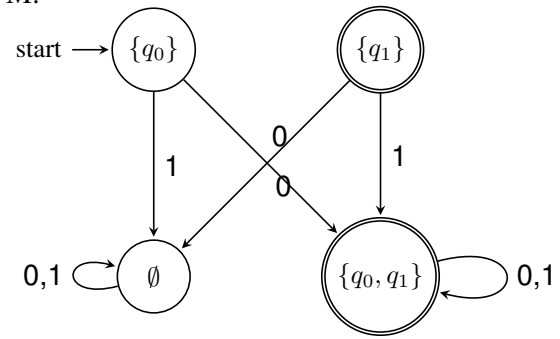
- $q'_0 = E(\{q_0\})$
- For $R \in Q'$, and $a \in \Sigma$, let $\delta'(R, a) = \{q \in Q | q \in E(\delta(r, a)) \text{ for some } r \in R\}$.

Example:

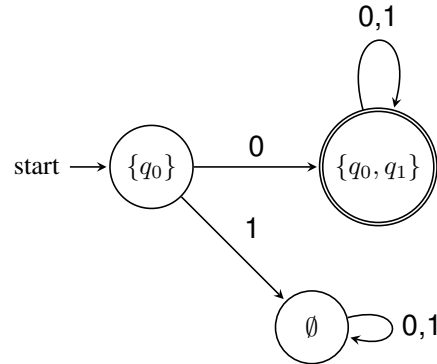
N:



M:

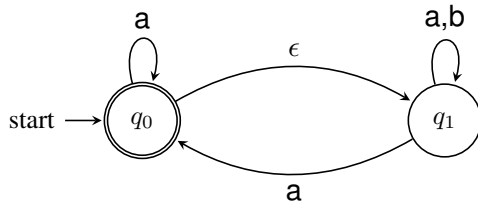


State $\{q_1\}$ is unreachable, therefore we discard this state. M becomes:

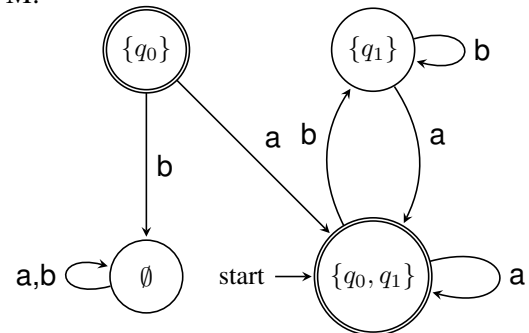


Example 2:

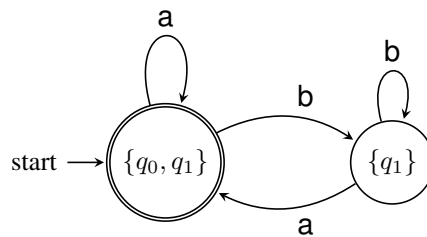
N:



M:



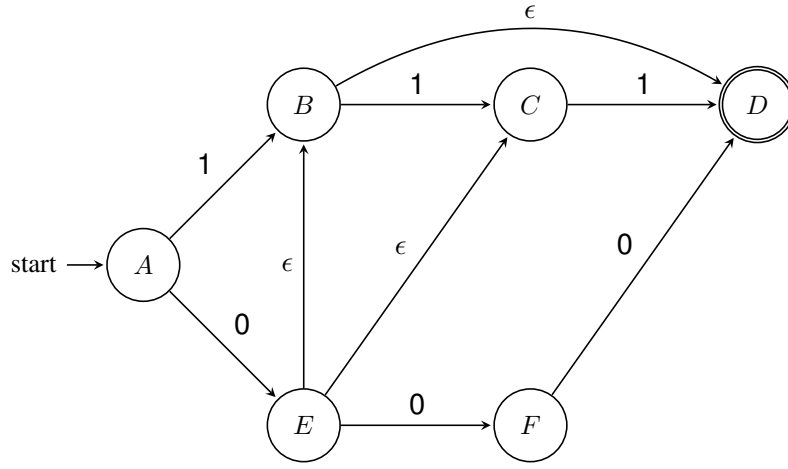
States $\{q_0\}$ and $\emptyset$ are unreachable, therefore we discard these states. M becomes:



$$L = \{w \mid w \in \{a, b\}^*, w \text{ ends with } a, \text{ or } w = \epsilon\}$$

Corollary 1.2 A language is regular iff it is accepted by some NFA.

1.1.2 ϵ -NFA to NFA



Closures:

$$\begin{aligned}
 E(\{A\}) &= \{A\} \\
 E(\{B\}) &= \{B, D\} \\
 E(\{C\}) &= \{C\} \\
 E(\{D\}) &= \{D\} \\
 E(\{E\}) &= \{B, C, D, E\} \\
 E(\{F\}) &= \{F\}
 \end{aligned}$$

ϵ -NFA	0	1	ϵ
$\rightarrow A$	$\{E\}$	$\{B\}$	$\emptyset$
B	$\emptyset$	$\{C\}$	$\{D\}$
C	$\emptyset$	$\{D\}$	$\emptyset$
*D	$\emptyset$	$\emptyset$	$\emptyset$
E	$\{F\}$	$\emptyset$	$\{B, C\}$
F	$\{D\}$	$\emptyset$	$\emptyset$

$\Rightarrow$

NFA	0	1
$\rightarrow A$	$\{E\}$	$\{B\}$
*B	$\emptyset$	$\{C\}$
C	$\emptyset$	$\{D\}$
*D	$\emptyset$	$\emptyset$
*E	$\{F\}$	$\{C, D\}$
F	$\{D\}$	$\emptyset$

States B and E become accepting states since their ϵ -closures include state D.

Assume δ is the transition function in ϵ -NFA, and δ' is the transition function in NFA. Then, for $q, r \in Q$, and $a \in \Sigma$:

$$\delta'(q, a) = \bigcup_{r \in E(q)} \delta(r, a)$$

- $\delta'(A, 0) = \bigcup_{r \in E(A)} \delta(r, 0) = \delta(A, 0) = \{E\}$
- $\delta'(A, 1) = \bigcup_{r \in E(A)} \delta(r, 1) = \delta(A, 1) = \{B\}$
- $\delta'(B, 0) = \bigcup_{r \in E(B)} \delta(r, 0) = \delta(B, 0) \cup \delta(D, 0) = \emptyset$
- $\delta'(B, 1) = \bigcup_{r \in E(B)} \delta(r, 1) = \delta(B, 1) \cup \delta(D, 1) = \{C\}$
- $\delta'(C, 0) = \bigcup_{r \in E(C)} \delta(r, 0) = \delta(C, 0) = \emptyset$
- $\delta'(C, 1) = \bigcup_{r \in E(C)} \delta(r, 1) = \delta(C, 1) = \{D\}$
- $\delta'(D, 0) = \bigcup_{r \in E(D)} \delta(r, 0) = \delta(D, 0) = \emptyset$
- $\delta'(D, 1) = \bigcup_{r \in E(D)} \delta(r, 1) = \delta(D, 1) = \emptyset$
- $\delta'(E, 0) = \bigcup_{r \in E(E)} \delta(r, 0) = \delta(B, 0) \cup \delta(C, 0) \cup \delta(D, 0) \cup \delta(E, 0) = \{F\}$
- $\delta'(E, 1) = \bigcup_{r \in E(E)} \delta(r, 1) = \delta(B, 1) \cup \delta(C, 1) \cup \delta(D, 1) \cup \delta(E, 1) = \{C, D\}$
- $\delta'(F, 0) = \bigcup_{r \in E(F)} \delta(r, 0) = \delta(F, 0) = \{D\}$
- $\delta'(F, 1) = \bigcup_{r \in E(F)} \delta(r, 1) = \delta(F, 1) = \emptyset$

1.2 Regular Operations

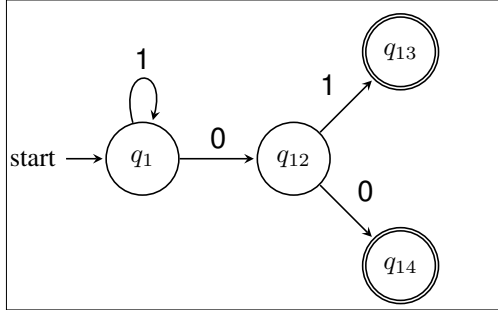
For languages L_1, L_2 :

- Union: $L_1 \cup L_2 = \{w | w \in L_1, \text{ or } w \in L_2\}$.
- Concatenation: $L_1 \cdot L_2 = \{w_1 w_2 | w_1 \in L_1, w_2 \in L_2\}$.
- Star ("Kleene closure"): $L_1^* = \{w_1 w_2 w_3 \dots w_k | k \geq 0, \text{ and } w_1, w_2, w_3, \dots, w_k \in L_1\}$.

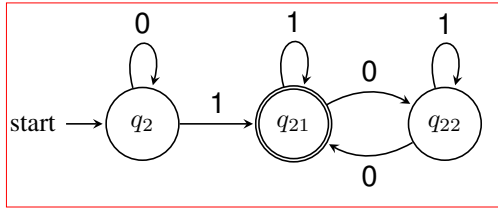
Remark $k = 0$ gives ϵ . Hence $\epsilon \in L^*$ for any L . If $L = \emptyset$, $L^* = \{\epsilon\}$.

Theorem 1.3 The class of regular languages are closed under regular operations. i.e. if L_1, L_2 are regular, then so are $L_1 \cup L_2$, $L_1 \cdot L_2$, and L^* .

Proof By construction. Assume L_1, L_2 are regular, and let M_1, M_2 be the DFAs that accept L_1, L_2 . We will construct NFA/DFA for $L_1 \cup L_2$, $L_1 \cdot L_2$, and L^* .

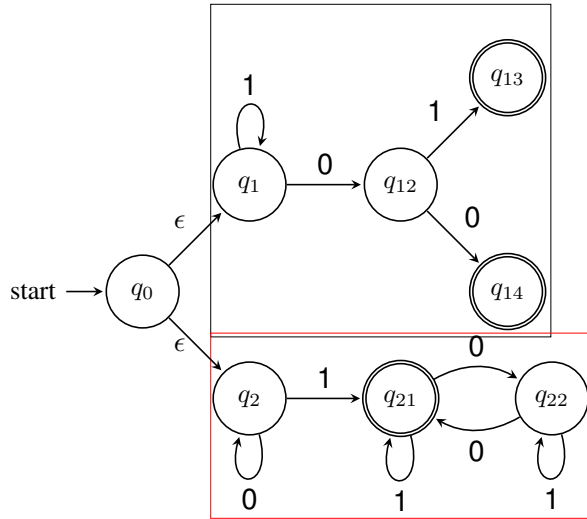


$$M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$$



$$M_2 = (Q_2, \Sigma, \delta_2, q_2, F)$$

- Union:



$$L(M) = L_1 \cup L_2.$$

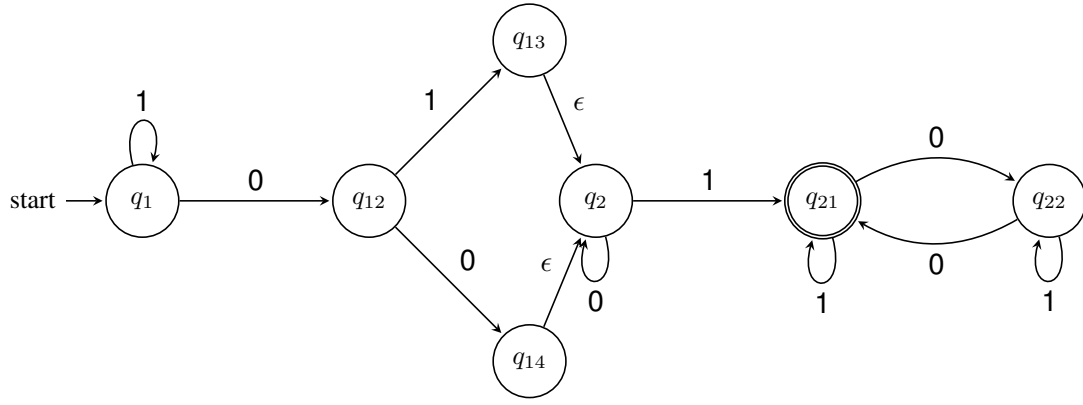
$$M = (Q, \Sigma, \delta, q_0, F) \text{ where}$$

$$- Q = Q_1 \cup Q_2 \cup q_0$$

$$- F = F_1 \cup F_2$$

$$- \delta(q, a) = \begin{cases} \delta_1(q, a), & \text{if } q \in Q_1 \\ \delta_2(q, a), & \text{if } q \in Q_2 \\ \{q_1, q_2\}, & \text{if } q = q_0, a = \epsilon \\ \emptyset, & \text{if } q = q_0, a \neq \epsilon \end{cases}$$

- Concatenation:

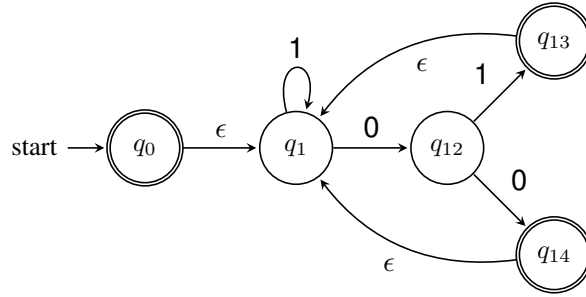


$$L(M) = L_1 \cdot L_2.$$

$M = (Q, \Sigma, \delta, q_1, F_2)$ where

- $Q = Q_1 \cup Q_2$
- $\delta(q, a) = \begin{cases} \delta_1(q, a), & \text{if } q \in Q_1 - F_1 \\ \delta_2(q, a), & \text{if } q \in Q_2 \\ \delta_1(q, a), & \text{if } q \in F_1, a \neq \epsilon \\ \delta_1(q, \epsilon) \cup \{q_2\}, & \text{if } q \in F_1, a = \epsilon \end{cases}$

- Star:



$$L(M) = L_1^*.$$

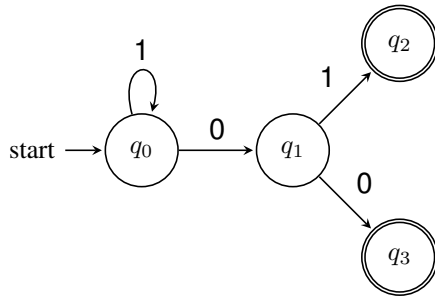
$M = (Q, \Sigma, \delta, q_0, F_2)$ where

- $Q = Q_1 \cup q_0$
- $F = F_1 \cup q_0$
- $\delta(q, a) = \begin{cases} \delta_1(q, a), & \text{if } q \in Q_1 - F_1 \\ \delta_1(q, a), & \text{if } q \in F_1, a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\}, & \text{if } q \in F_1, a = \epsilon \\ \{q_1\}, & \text{if } q = q_0 \text{ and } a = \epsilon \\ \emptyset, & \text{if } q = q_0 \text{ and } a \neq \epsilon \end{cases}$

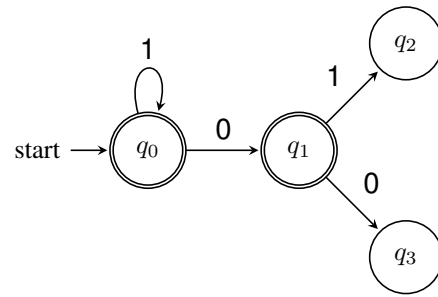
Theorem 1.4 *The class of regular languages are closed under complementation. i.e. if $L \subseteq \Sigma^*$ is regular, then so is $\bar{L} = \Sigma^* - L$.*

Proof By construction. Assume L is a regular language (RL) accepted by some DFA $M = (Q, \Sigma, \delta, q_0, F)$. Then $\bar{L}$ is accepted by $M' = (Q, \Sigma, \delta, q_0, Q - F)$.

M :

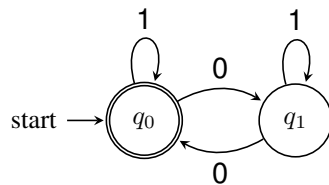


M' :

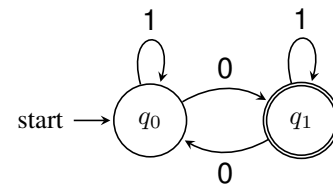


Example:

$L = \{w \mid w \text{ has an even number of 0s}\}$

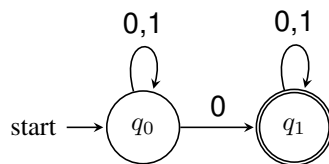


$L = \{w \mid w \text{ has an odd number of 0s}\}$

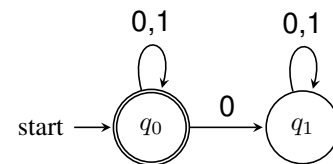


We cannot take complement of an NFA. Turn it into DFA, then complement. For example, $w = 00$ is accepted by both N and N' :

N :



N' :



Corollary 1.5 *Regular languages are closed under*

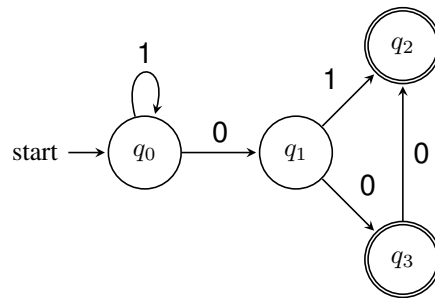
- *Intersection:* $L_1 \cap L_2$. Note that $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.
- *Set difference:* $L_1 - L_2$. Note that $L_1 - L_2 = L_1 \cap \overline{L_2}$

1.2.1 Reversal Operation

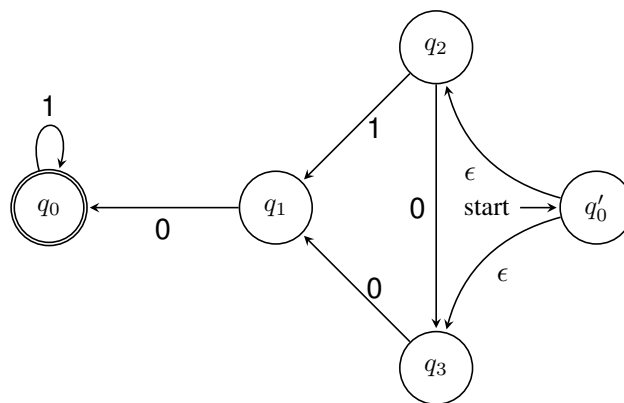
- For a string w , its reverse w^R is w written in reverse order.
eg. $w = 0010$, $w^R = 0100$.
- For a language L , its reverse L^R is defined as:
 $L^R = \{w^R \mid w \in L\}$. e.g.
 - $L = \{01, 0010\}$, $L^R = \{10, 0100\}$
 - $L = \{w \mid w \text{ ends with } 0\}$, $L^R = \{w \mid w \text{ starts with } 0\}$

Theorem 1.6 *RLs are closed under the reversal operation.*

Proof Let L be an RL accepted by $M = (Q, \Sigma, \delta, q_0, F)$:



For L^R , take the same states, and revert the transitions. Also, make the start state an accept state. Make the accept states the start state. Since this may mean multiple start states, add a new start state and link it to the accept states of M by ϵ -transitions.

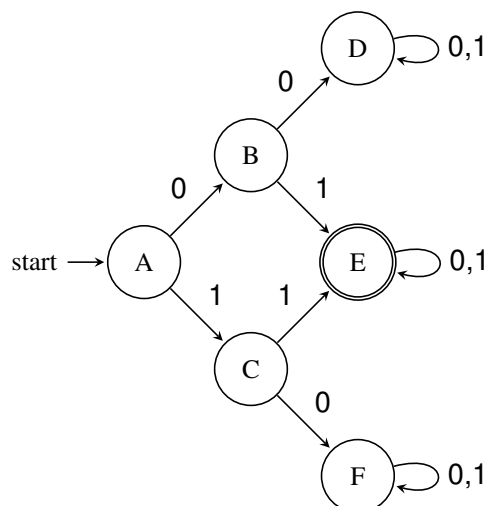


1.3 Equivalence and Minimization of DFA

- How can we tell whether two DFAs M_1 and M_2 are equivalent?
- Is there a “minimal DFA” for a given RL?
- If there is, how can we find it?

Equivalent states: We say two states p and q are “equivalent” (or “indistinguishable”) if they return the same result (accept/reject) for every string w .

M :



In M , states D and F are equivalent: $D \equiv F$.

In M , states B and C are equivalent: $B \equiv C$.

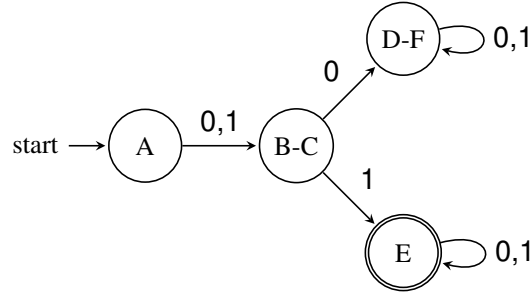
Definition: Two states p and q are equivalent/indistinguishable if

$$\delta^*(p, w) \in F \Leftrightarrow \delta^*(q, w) \in F \text{ for all } w \in \Sigma^*.$$

Else, if for some w , $\delta^*(p, w) \in F$ while $\delta^*(q, w) \notin F$, or vice versa, we say p & q are “distinguishable” by string w .

Once we identify the equivalent states, we can minimize the DFA by merging them together.

M' :



$$L(M) = L(M') = \{w \mid \text{the second character of } w \text{ is } 1\}$$

Remark An accept & a non-accept state are always distinguishable (by string ϵ).

Algorithm 1 Equivalent states

Initialization: For every pair of states (p, q) where one is accept and the other is not, mark (p, q) as distinguishable.

markFlag $\leftarrow$ true

while markFlag = true **do**

markFlag $\leftarrow$ false

for every unmarked pair p, q **do**

for every symbol $a \in \Sigma$ **do**

if the states $r = \delta(p, a)$ and $s = \delta(q, a)$ are marked as distinguishable **then**

mark (p, q) as distinguishable

markFlag $\leftarrow$ true

break

end if

end for

end for

end while

Example: Using M from above:

	A	B	C	D	E	F
A						
B	x^1					
C	x^1	✓				
D	x^2	x^1	x^1			
E	x^0	x^0	x^0	x^0		
F	x^2	x^1	x^1	✓	x^0	

x^0 : Initialization

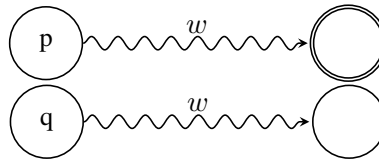
x^1 : First iteration

x^2 : Second iteration

✓: equivalent $\Rightarrow$

In the third iteration there were no newly marked pairs, so we stop and decide all non-marked ones are equivalent.

Correctness of the algorithm: if some pair (p, q) is marked as distinguishable by the algorithm, then for some w :



or vice versa. Conversely, if some pair (p, q) was not marked by the algorithm, then they cannot be distinguishable.

If p & q could be distinguishable by some w , the algorithm would not stop before marking (p, q) as distinguishable.

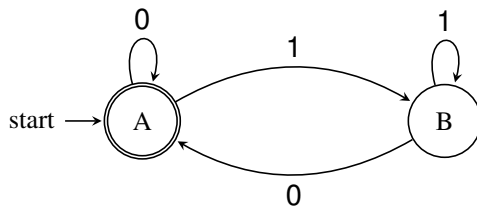
1.3.1 Equivalence of DFA

Given two DFAs M_1 and M_2 :

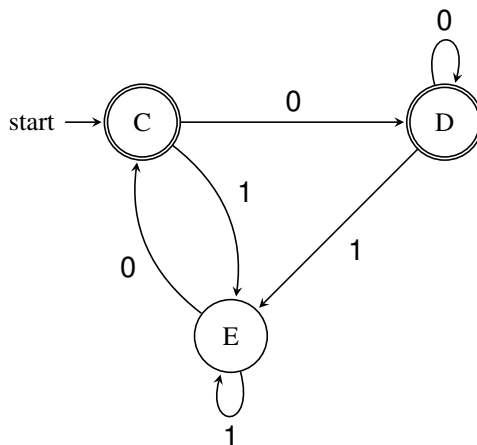
- Take M_1, M_2 together.
- Test for equivalent states.
- If the start states of M_1 & M_2 turn out to be equivalent, then $L(M_1) = L(M_2)$ and $M_1 = M_2$.

e.g.

M_1 :



M_2 :



	A	B	C	D
B	×			
C	✓	×		
D	✓	×	✓	
E	×	✓	×	×

$$A \equiv C \equiv D$$

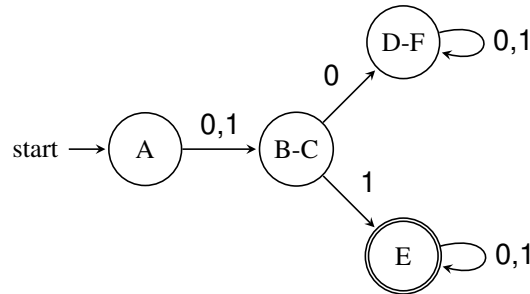
$$B \equiv E$$

$$\Rightarrow M_1 \equiv M_2$$

1.3.2 Minimization of DFA

- First, remove any states not reachable from the start state.
- Run the equivalent state detection algorithm. Group equivalent states together.

Eg. From DFA M , assume we found that $B \equiv C$ and $D \equiv F$:



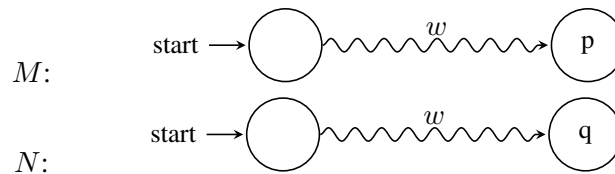
Question: For some minimized DFA M , can there be an equivalent DFA with fewer states?

Answer: No.

Theorem 1.7 *If M is a DFA minimized by the procedure above, then any equivalent DFA cannot have fewer states than M .*

Proof By contradiction. Let M be a DFA obtained by the minimization algorithm, and assume N is an equivalent DFA with fewer states.

- For every string w , the state in M reached by w is equivalent to the state in N reached by w .



- Since M is a minimized DFA, every state in M is reachable, hence every state in M is equivalent to some state in N .
- Since we assume that N has fewer states than M , by the pigeonhole principle, M should have at least two states p_1 and p_2 which are equivalent to the same state q in N :

$$\begin{aligned}
 p_1 &\equiv q \\
 p_2 &\equiv q \\
 \text{then, } p_1 &\equiv p_2
 \end{aligned}$$

- This is a contradiction with the fact that M is a minimized DFA. Hence, no such N can exist.

Corollary 1.8 *For a regular language, there is a unique “minimal DFA”. This is the DFA obtained by the minimization algorithm.*

Proof Follows from the observation in the proof of the previous theorem (Theorem 1.7) that there is one-to-one correspondence between the states of two minimized DFA.